LIN7076 – Foundations of Computational Linguistics

Regular Expressions and Automata

Adèle Hénot-Mortier

9/11/2025

Queen Mary University of London

Deep intuitions about well-formedness

(1) a. Colorless green ideas sleep furiously.

b. * Furiously colorless green sleep ideas.

'Twas brillig, and the slithy toves
Did gyre and gimble in the wabe:
All mimsy were the borogoves,
And the mome raths outgrabe.
"Beware the Jabberwock, my son!
The jaws that bite, the claws that catch!
Beware the Jubjub bird, and shun
The frumious Bandersnatch!"

Jabberwocky, Lewis Carroll

Plan for today

Can we come up with simple rules summarizing a language or a fragment thereof?

- We'll take a break from statistics and algebra to talk a bit about **formalizing language structure**.
- We'll explore two main approaches: **regular expressions** (this week), **context-free grammars** (next week).
- We'll discuss how these can be used for concrete applications: find-and-replace, parsing and structure-mining.
- We'll also review some aspects of natural language that are fundamentally challenging for these approaches.

Plan for today

Can we come up with simple rules summarizing a language or a fragment thereof?

- We'll take a break from statistics and algebra to talk a bit about **formalizing language structure**.
- We'll explore two main approaches: **regular expressions** (this week), **context-free grammars** (next week).
- We'll discuss how these can be used for concrete applications: find-and-replace, parsing and structure-mining.
- We'll also review some aspects of natural language that are fundamentally challenging for these approaches.

Plan for today

Can we come up with simple rules summarizing a language or a fragment thereof?

- We'll take a break from statistics and algebra to talk a bit about **formalizing language structure**.
- We'll explore two main approaches: **regular expressions** (this week), **context-free grammars** (next week).
- We'll discuss how these can be used for concrete applications: find-and-replace, parsing and structure-mining.
- We'll also review some aspects of natural language that are fundamentally challenging for these approaches.

Plan for today

Can we come up with simple rules summarizing a language or a fragment thereof?

- We'll take a break from statistics and algebra to talk a bit about **formalizing language structure**.
- We'll explore two main approaches: **regular expressions** (this week), **context-free grammars** (next week).
- We'll discuss how these can be used for concrete applications: find-and-replace, parsing and structure-mining.
- We'll also review some aspects of natural language that are fundamentally challenging for these approaches.

Plan for today

Can we come up with simple rules summarizing a language or a fragment thereof?

- We'll take a break from statistics and algebra to talk a bit about **formalizing language structure**.
- We'll explore two main approaches: **regular expressions** (this week), **context-free grammars** (next week).
- We'll discuss how these can be used for concrete applications: find-and-replace, parsing and structure-mining.
- We'll also review some aspects of natural language that are fundamentally challenging for these approaches.

Examples of ungrammaticality/ill-formedness

- Tensed embedded clause cannot be subject-less ("ban on hyperraising"):

(2) a. * Jo seems [that won]. b. Jo seems [to have won].

- Subjects and main verbs must agree:

(3) a. * The dogs barks. b. The dog barks.

- Adverbs must precede main verbs in English:

(4) a. * Jo eats often apples. b. Jo often eats apples.

- All these generalizations are independent of meaning–they are **syntactic**, and sometimes language-dependent.
- Violating them causes ungrammaticality/**ill-formedness**.

Examples of ungrammaticality/ill-formedness

- Tensed embedded clause cannot be subject-less ("ban on hyperraising"):

(2) a. * Jo seems [that won]. b. Jo seems [to have won].

- Subjects and main verbs must agree:

(3) a. * The dogs barks. b. The dog barks.

- Adverbs must precede main verbs in English:

(4) a. * Jo eats often apples. b. Jo often eats apples.

- All these generalizations are independent of meaning–they are **syntactic**, and sometimes language-dependent.
- Violating them causes ungrammaticality/**ill-formedness**.

Examples of ungrammaticality/ill-formedness

- Tensed embedded clause cannot be subject-less ("ban on hyperraising"):

(2) a. * Jo seems [that won]. b. Jo seems [to have won].

- Subjects and main verbs must agree:

(3) a. * The dogs barks. b. The dog barks.

- Adverbs must precede main verbs in English:

(4) a. * Jo eats often apples. b. Jo often eats apples.

- All these generalizations are independent of meaning–they are **syntactic**, and sometimes language-dependent.
- Violating them causes ungrammaticality/**ill-formedness**.

Examples of ungrammaticality/ill-formedness

- Tensed embedded clause cannot be subject-less ("ban on hyperraising"):

(2) a. * Jo seems [that won]. b. Jo seems [to have won].

- Subjects and main verbs must agree:

(3) a. * The dogs barks. b. The dog barks.

- Adverbs must precede main verbs in English:

(4) a. * Jo eats often apples. b. Jo often eats apples.

- All these generalizations are independent of meaning–they are **syntactic**, and sometimes language-dependent.
- Violating them causes ungrammaticality/**ill-formedness**.

Examples of ungrammaticality/ill-formedness

- Tensed embedded clause cannot be subject-less ("ban on hyperraising"):

(2) a. * Jo seems [that won]. b. Jo seems [to have won].

- Subjects and main verbs must agree:

(3) a. * The dogs barks. b. The dog barks.

- Adverbs must precede main verbs in English:

(4) a. * Jo eats often apples. b. Jo often eats apples.

- All these generalizations are independent of meaning–they are **syntactic**, and sometimes language-dependent.
- Violating them causes ungrammaticality/**ill-formedness**.

Examples of infelicity

- A proposition and its negation cannot be conjoined (contradictory!) or disjoined (tautology!):

(5) a. # Jo is inside and outside.

b. # Jo is inside or outside.

- Sentences should take obvious facts for granted...

(6) a. # A sun is shining.
⇝ There's at least one sun.

b. The sun is shining.
⇝ There's a unique sun.

- But should not take new information for granted:

(7) a. ?? Jo will need to bring **her alligator** to the vet.

b. Jo **has an alligator**, and will need to bring it to the vet.

- These generalizations depend on meaning (**semantic**) and largely language-independent. Violating them causes **infelicity**.

Examples of infelicity

- A proposition and its negation cannot be conjoined (contradictory!) or disjoined (tautology!):

(5) a. # Jo is inside and outside.

b. # Jo is inside or outside.

- Sentences should take obvious facts for granted...

(6) a. # A sun is shining.
⇝ There's at least one sun.

b. The sun is shining.
⇝ There's a unique sun.

- But should not take new information for granted:

(7) a. ?? Jo will need to bring **her alligator** to the vet.

b. Jo **has an alligator**, and will need to bring it to the vet.

- These generalizations depend on meaning (**semantic**) and largely language-independent. Violating them causes **infelicity**.

Examples of infelicity

- A proposition and its negation cannot be conjoined (contradictory!) or disjoined (tautology!):

(5) a. # Jo is inside and outside.

b. # Jo is inside or outside.

- Sentences should take obvious facts for granted…

(6) a. # A sun is shining.
⇝ There's at least one sun.

b. The sun is shining.
⇝ There's a unique sun.

- But should not take new information for granted:

(7) a. ?? Jo will need to bring **her alligato**r to the vet.

b. Jo **has an alligator**, and will need to bring it to the vet.

- These generalizations depend on meaning (**semantic**) and largely language-independent. Violating them causes **infelicity**.

Examples of infelicity

- A proposition and its negation cannot be conjoined (contradictory!) or disjoined (tautology!):

(5) a. # Jo is inside and outside.

b. # Jo is inside or outside.

- Sentences should take obvious facts for granted...

(6) a. # A sun is shining.
⇝ There's at least one sun.

b. The sun is shining.
⇝ There's a unique sun.

- But should not take new information for granted:

(7) a. ?? Jo will need to bring **her alligato**r to the vet.

b. Jo **has an alligator**, and will need to bring it to the vet.

- These generalizations depend on meaning (**semantic**) and largely language-independent. Violating them causes **infelicity**.

Examples of infelicity

- A proposition and its negation cannot be conjoined (contradictory!) or disjoined (tautology!):

(5) a. # Jo is inside and outside.

b. # Jo is inside or outside.

- Sentences should take obvious facts for granted...

(6) a. # A sun is shining.
⇝ There's at least one sun.

b. The sun is shining.
⇝ There's a unique sun.

- But should not take new information for granted:

(7) a. ?? Jo will need to bring **her alligato**r to the vet.

b. Jo **has an alligator**, and will need to bring it to the vet.

- These generalizations depend on meaning (**semantic**) and largely language-independent. Violating them causes **infelicity**.

Formalizing well-formedness

- Today we'll **focus on modeling grammaticality/well-formedness**.
- A language can be modeled as **the set of all the sentences that are well-formed in that language–independently of meaning**.
- For instance, English = {The dog barks, Colorless green ideas sleep furiously, ...}. How big is this set?

Formalizing well-formedness

- Today we'll **focus on modeling grammaticality/well-formedness**.
- A language can be modeled as **the set of all the sentences that are well-formed in that language–independently of meaning**.
- For instance, English = {The dog barks, Colorless green ideas sleep furiously, ...}. How big is this set?

Formalizing well-formedness

- Today we'll **focus on modeling grammaticality/well-formedness**.
- A language can be modeled as **the set of all the sentences that are well-formed in that language–independently of meaning**.
- For instance, English = {The dog barks, Colorless green ideas sleep furiously, ...}. How big is this set?

Other possible specifications of a “language”

- More abstractly, a language can be seen as the set of well-formed sequences of syntactic categories (instead of words instantiating these categories).
    - In that case, English = {Det Noun Verb, Adj Adj Noun Verb Adv, ...}. How big is this set?
- More narrowly, a language can be seen as the set of well-formed sequences of phonological objects (e.g. consonants C, and vowels V[1])
    - English is hard to illustrate, so let’s take Japanese whose building blocks are mostly CV syllables plus occasional nasals. In that case Japanese = {CV, CVCV, CVCVCV, CVnCV, CVCVn, ...}.
- In all these cases, we are interested in **well-formed units of a language** (strings of words; strings of syntactic labels; strings of phonemes...).

[1] One can choose more specific categories too!

Other possible specifications of a “language”

- More abstractly, a language can be seen as the set of well-formed sequences of syntactic categories (instead of words instantiating these categories).
  - In that case, English = {Det Noun Verb, Adj Adj Noun Verb Adv, ...}. How big is this set?
- More narrowly, a language can be seen as the set of well-formed sequences of phonological objects (e.g. consonants C, and vowels V[1])
  - English is hard to illustrate, so let’s take Japanese whose building blocks are mostly CV syllables plus occasional nasals. In that case Japanese = {CV, CVCV, CVCVCV, CVnCV, CVCVn, ...}.
- In all these cases, we are interested in **well-formed units of a language** (strings of words; strings of syntactic labels; strings of phonemes...).

[1] One can choose more specific categories too!

Other possible specifications of a “language”

- More abstractly, a language can be seen as the set of well-formed sequences of syntactic categories (instead of words instantiating these categories).
  - In that case, English = {Det Noun Verb, Adj Adj Noun Verb Adv, ...}. How big is this set?
- More narrowly, a language can be seen as the set of well-formed sequences of phonological objects (e.g. consonants C, and vowels V[1])
  - English is hard to illustrate, so let’s take Japanese whose building blocks are mostly CV syllables plus occasional nasals. In that case Japanese = {CV, CVCV, CVCVCV, CVnCV, CVCVn, ...}.
- In all these cases, we are interested in **well-formed units of a language** (strings of words; strings of syntactic labels; strings of phonemes...).

[1] One can choose more specific categories too!

Other possible specifications of a “language”

- More abstractly, a language can be seen as the set of well-formed sequences of syntactic categories (instead of words instantiating these categories).
  - In that case, English = {Det Noun Verb, Adj Adj Noun Verb Adv, ...}. How big is this set?
- More narrowly, a language can be seen as the set of well-formed sequences of phonological objects (e.g. consonants C, and vowels V[1])
  - English is hard to illustrate, so let’s take Japanese whose building blocks are mostly CV syllables plus occasional nasals. In that case Japanese = {CV, CVCV, CVCVCV, CVnCV, CVCVn, ...}.
- In all these cases, we are interested in **well-formed units of a language** (strings of words; strings of syntactic labels; strings of phonemes...).

[1] One can choose more specific categories too!

Other possible specifications of a “language”

- More abstractly, a language can be seen as the set of well-formed sequences of syntactic categories (instead of words instantiating these categories).
  - In that case, English = {Det Noun Verb, Adj Adj Noun Verb Adv, ...}. How big is this set?
- More narrowly, a language can be seen as the set of well-formed sequences of phonological objects (e.g. consonants C, and vowels V[1])
  - English is hard to illustrate, so let’s take Japanese whose building blocks are mostly CV syllables plus occasional nasals. In that case Japanese = {CV, CVCV, CVCVCV, CVnCV, CVCVn, ...}.
- In all these cases, we are interested in **well-formed units of a language** (strings of words; strings of syntactic labels; strings of phonemes...).

[1] One can choose more specific categories too!

The need for recursivity

- To model natural languages, formal languages need to generate **an infinite number of outputs from finite means**.
- Finite means:
  - A **finite alphabet** Σ of primitive "words" (lexical items, or syntactic categories).
  - A **finite set of rules** with some degree of **recursivity**.
- Recursivity is the **capacity for a rule to refer to itself**. Because recursive rules may apply an arbitrary number of times, an infinity of outputs can be generated.
- Concrete example: a folder-based file system is inherently recursive. Any **folder** may contain only **files** ("base case"), or, at least one other **folder** ("recursive case").
- More generally, anything that displays an **arborescent structure** will be recursive!

The need for recursivity

- To model natural languages, formal languages need to generate **an infinite number of outputs from finite means**.
- Finite means:
  - A **finite alphabet Σ** of primitive “words” (lexical items, or syntactic categories).
  - A **finite set of rules** with some degree of **recursivity**.
- Recursivity is the **capacity for a rule to refer to itself**. Because recursive rules may apply an arbitrary number of times, an infinity of outputs can be generated.
- Concrete example: a folder-based file system is inherently recursive. Any **folder** may contain only **files** (“base case”), or, at least one other **folder** (“recursive case”).
- More generally, anything that displays an **arborescent structure** will be recursive!

The need for recursivity

- To model natural languages, formal languages need to generate **an infinite number of outputs from finite means**.
- Finite means:
  - A **finite alphabet Σ** of primitive “words” (lexical items, or syntactic categories).
  - A **finite set of rules** with some degree of **recursivity**.
- Recursivity is the **capacity for a rule to refer to itself**. Because recursive rules may apply an arbitrary number of times, an infinity of outputs can be generated.
- Concrete example: a folder-based file system is inherently recursive. Any **folder** may contain only **files** (“base case”), or, at least one other **folder** (“recursive case”).
- More generally, anything that displays an **arborescent structure** will be recursive!

The need for recursivity

- To model natural languages, formal languages need to generate **an infinite number of outputs from finite means**.
- Finite means:
  - A **finite alphabet Σ** of primitive “words” (lexical items, or syntactic categories).
  - A **finite set of rules** with some degree of **recursivity**.
- Recursivity is the **capacity for a rule to refer to itself**. Because recursive rules may apply an arbitrary number of times, an infinity of outputs can be generated.
- Concrete example: a folder-based file system is inherently recursive. Any **folder** may contain only **files** (“base case”), or, at least one other **folder** (“recursive case”).
- More generally, anything that displays an **arborescent structure** will be recursive!

The need for recursivity

- To model natural languages, formal languages need to generate **an infinite number of outputs from finite means**.
- Finite means:
  - A **finite alphabet** Σ of primitive “words” (lexical items, or syntactic categories).
  - A **finite set of rules** with some degree of **recursivity**.
- Recursivity is the **capacity for a rule to refer to itself**. Because recursive rules may apply an arbitrary number of times, an infinity of outputs can be generated.
- Concrete example: a folder-based file system is inherently recursive. Any **folder** may contain only **files** (“base case”), or, at least one other **folder** (“recursive case”).
- More generally, anything that displays an **arborescent structure** will be recursive!

The need for recursivity

- To model natural languages, formal languages need to generate **an infinite number of outputs from finite means**.
- Finite means:
  - A **finite alphabet** Σ of primitive "words" (lexical items, or syntactic categories).
  - A **finite set of rules** with some degree of **recursivity**.
- Recursivity is the **capacity for a rule to refer to itself**. Because recursive rules may apply an arbitrary number of times, an infinity of outputs can be generated.
- Concrete example: a folder-based file system is inherently recursive. Any **folder** may contain only **files** ("base case"), or, at least one other **folder** ("recursive case").
- More generally, anything that displays an **arborescent structure** will be recursive!

A toy example: intensified adjectival phrases

- Suppose our alphabet Σ is { a, the, very, tall, girl, boy }. What kind of well-formed phrases can we build out of this alphabet?
  - *the boy* and *the girl* are well-formed.
  - *a boy* and *a girl* are well-formed.
  - *the/a tall boy/girl* are well-formed.
  - *the/a very tall boy/girl* are well-formed.
  - *the/a very very tall boy/girl* are well-formed.
  - *the/a very very very tall boy/girl* are well-formed.
  - ...
- Where do we spot recursivity here?

A toy example: intensified adjectival phrases

- Suppose our alphabet Σ is { a, the, very, tall, girl, boy }. What kind of well-formed phrases can we build out of this alphabet?
  - *the boy* and *the girl* are well-formed.
  - *a boy* and *a girl* are well-formed.
  - *the/a tall boy/girl* are well-formed.
  - *the/a very tall boy/girl* are well-formed.
  - *the/a very very tall boy/girl* are well-formed.
  - *the/a very very very tall boy/girl* are well-formed.
  - ...
- Where do we spot recursivity here?

A toy example: intensified adjectival phrases

- Suppose our alphabet Σ is { a, the, very, tall, girl, boy }. What kind of well-formed phrases can we build out of this alphabet?
  - *the boy* and *the girl* are well-formed.
  - *a boy* and *a girl* are well-formed.
  - *the/a tall boy/girl* are well-formed.
  - *the/a very tall boy/girl* are well-formed.
  - *the/a very very tall boy/girl* are well-formed.
  - *the/a very very very tall boy/girl* are well-formed.
  - ...
- Where do we spot recursivity here?

A toy example: intensified adjectival phrases

- Suppose our alphabet Σ is { a, the, very, tall, girl, boy }. What kind of well-formed phrases can we build out of this alphabet?
  - *the boy* and *the girl* are well-formed.
  - *a boy* and *a girl* are well-formed.
  - *the/a tall boy/girl* are well-formed.
  - *the/a very tall boy/girl* are well-formed.
  - *the/a very very tall boy/girl* are well-formed.
  - *the/a very very very tall boy/girl* are well-formed.
  - ...
- Where do we spot recursivity here?

A toy example: intensified adjectival phrases

- Suppose our alphabet Σ is { a, the, very, tall, girl, boy }. What kind of well-formed phrases can we build out of this alphabet?
  - *the boy* and *the girl* are well-formed.
  - *a boy* and *a girl* are well-formed.
  - *the/a tall boy/girl* are well-formed.
  - *the/a very tall boy/girl* are well-formed.
  - *the/a very very tall boy/girl* are well-formed.
  - *the/a very very very tall boy/girl* are well-formed.
  - ...
- Where do we spot recursivity here?

A toy example: intensified adjectival phrases

- Suppose our alphabet Σ is { a, the, very, tall, girl, boy }. What kind of well-formed phrases can we build out of this alphabet?
  - *the boy* and *the girl* are well-formed.
  - *a boy* and *a girl* are well-formed.
  - *the/a tall boy/girl* are well-formed.
  - *the/a very tall boy/girl* are well-formed.
  - *the/a very very tall boy/girl* are well-formed.
  - ...
- Where do we spot recursivity here?

A toy example: intensified adjectival phrases

- Suppose our alphabet Σ is { a, the, very, tall, girl, boy }. What kind of well-formed phrases can we build out of this alphabet?
  - *the boy* and *the girl* are well-formed.
  - *a boy* and *a girl* are well-formed.
  - *the/a tall boy/girl* are well-formed.
  - *the/a very tall boy/girl* are well-formed.
  - *the/a very very tall boy/girl* are well-formed.
  - *the/a very very very tall boy/girl* are well-formed.
  - ...
- Where do we spot recursivity here?

A toy example: intensified adjectival phrases

- Suppose our alphabet Σ is { a, the, very, tall, girl, boy }. What kind of well-formed phrases can we build out of this alphabet?
  - *the boy* and *the girl* are well-formed.
  - *a boy* and *a girl* are well-formed.
  - *the/a tall boy/girl* are well-formed.
  - *the/a very tall boy/girl* are well-formed.
  - *the/a very very tall boy/girl* are well-formed.
  - *the/a very very very tall boy/girl* are well-formed.
  - ...
- Where do we spot recursivity here?

A toy example: intensified adjectival phrases

- Suppose our alphabet Σ is { a, the, very, tall, girl, boy }. What kind of well-formed phrases can we build out of this alphabet?
  - *the boy* and *the girl* are well-formed.
  - *a boy* and *a girl* are well-formed.
  - *the/a tall boy/girl* are well-formed.
  - *the/a very tall boy/girl* are well-formed.
  - *the/a very very tall boy/girl* are well-formed.
  - *the/a very very very tall boy/girl* are well-formed.
  - ...
- Where do we spot recursivity here?

Regular expressions

- Let Σ be an alphabet. We call the members of Σ **words** (they can be "real" words like *cat*, syntactic labels like Det, phonological units like V...).
- Regular expressions (**regexes**) are used to specify **word patterns**.
- Regexes are built out of consist of **constants**, which denote sets of words, and **operator symbols**, which denote operations over these sets.
- Constants: $\emptyset$ (the empty regex), and $\{\epsilon\}$ (the empty-word regex) are regexes. For any w in Σ, $\{w\}$ is a regex.
- Operations: given R and S two regexes...
  - RS is the pairwise **concatenation** of R and S and is a regex.
  - $R|S$ is the **union** of R and S and is a regex.
  - R^* is the set containing ϵ and all possible concatenations of elements of R ("**Kleene star**") and is a regex.
- We'll use parentheses to signal the order of precedence between these operations.

Regular expressions

- Let Σ be an alphabet. We call the members of Σ **words** (they can be "real" words like *cat*, syntactic labels like Det, phonological units like V...).
- Regular expressions (**regexes**) are used to specify **word patterns**.
- Regexes are built out of consist of **constants**, which denote sets of words, and **operator symbols**, which denote operations over these sets.
- Constants: $\emptyset$ (the empty regex), and $\{\epsilon\}$ (the empty-word regex) are regexes. For any w in Σ, $\{w\}$ is a regex.
- Operations: given R and S two regexes...
  - RS is the pairwise **concatenation** of R and S and is a regex.
  - $R|S$ is the **union** of R and S and is a regex.
  - R^* is the set containing ϵ and all possible concatenations of elements of R ("**Kleene star**") and is a regex.
- We'll use parentheses to signal the order of precedence between these operations.

Regular expressions

- Let Σ be an alphabet. We call the members of Σ **words** (they can be "real" words like *cat*, syntactic labels like Det, phonological units like V...).
- Regular expressions (**regexes**) are used to specify **word patterns**.
- Regexes are built out of consist of **constants**, which denote sets of words, and **operator symbols**, which denote operations over these sets.
- Constants: $\emptyset$ (the empty regex), and $\{\epsilon\}$ (the empty-word regex) are regexes. For any w in Σ, $\{w\}$ is a regex.
- Operations: given R and S two regexes...
  - RS is the pairwise **concatenation** of R and S and is a regex.
  - $R|S$ is the **union** of R and S and is a regex.
  - R^* is the set containing ϵ and all possible concatenations of elements of R ("**Kleene star**") and is a regex.
- We'll use parentheses to signal the order of precedence between these operations.

Regular expressions

- Let Σ be an alphabet. We call the members of Σ **words** (they can be "real" words like *cat*, syntactic labels like Det, phonological units like V...).
- Regular expressions (**regexes**) are used to specify **word patterns**.
- Regexes are built out of consist of **constants**, which denote sets of words, and **operator symbols**, which denote operations over these sets.
- Constants: $\emptyset$ (the empty regex), and $\{\epsilon\}$ (the empty-word regex) are regexes. For any w in Σ, $\{w\}$ is a regex.
- Operations: given R and S two regexes...
  - RS is the pairwise **concatenation** of R and S and is a regex.
  - $R|S$ is the **union** of R and S and is a regex.
  - R^* is the set containing ϵ and all possible concatenations of elements of R ("**Kleene star**") and is a regex.
- We'll use parentheses to signal the order of precedence between these operations.

Regular expressions

- Let Σ be an alphabet. We call the members of Σ **words** (they can be "real" words like *cat*, syntactic labels like Det, phonological units like V...).
- Regular expressions (**regexes**) are used to specify **word patterns**.
- Regexes are built out of consist of **constants**, which denote sets of words, and **operator symbols**, which denote operations over these sets.
- Constants: $\emptyset$ (the empty regex), and $\{\epsilon\}$ (the empty-word regex) are regexes. For any w in Σ, $\{w\}$ is a regex.
- Operations: given R and S two regexes...
  - RS is the pairwise **concatenation** of R and S and is a regex.
  - $R|S$ is the **union** of R and S and is a regex.
  - R^* is the set containing ϵ and all possible concatenations of elements of R ("**Kleene star**") and is a regex.
- We'll use parentheses to signal the order of precedence between these operations.

Regular expressions

- Let Σ be an alphabet. We call the members of Σ **words** (they can be "real" words like *cat*, syntactic labels like Det, phonological units like V...).
- Regular expressions (**regexes**) are used to specify **word patterns**.
- Regexes are built out of consist of **constants**, which denote sets of words, and **operator symbols**, which denote operations over these sets.
- Constants: $\emptyset$ (the empty regex), and $\{\epsilon\}$ (the empty-word regex) are regexes. For any w in Σ, $\{w\}$ is a regex.
- Operations: given R and S two regexes...
  - RS is the pairwise **concatenation** of R and S and is a regex.
  - $R|S$ is the **union** of R and S and is a regex.
  - R^* is the set containing ϵ and all possible concatenations of elements of R ("**Kleene star**") and is a regex.
- We'll use parentheses to signal the order of precedence between these operations.

Regular expressions

- Let Σ be an alphabet. We call the members of Σ **words** (they can be "real" words like *cat*, syntactic labels like Det, phonological units like V...).
- Regular expressions (**regexes**) are used to specify **word patterns**.
- Regexes are built out of consist of **constants**, which denote sets of words, and **operator symbols**, which denote operations over these sets.
- Constants: $\emptyset$ (the empty regex), and $\{\epsilon\}$ (the empty-word regex) are regexes. For any w in Σ, $\{w\}$ is a regex.
- Operations: given R and S two regexes...
  - RS is the pairwise **concatenation** of R and S and is a regex.
  - $R|S$ is the **union** of R and S and is a regex.
  - R^* is the set containing ϵ and all possible concatenations of elements of R ("**Kleene star**") and is a regex.
- We'll use parentheses to signal the order of precedence between these operations.

Regular expressions

- Let Σ be an alphabet. We call the members of Σ **words** (they can be "real" words like *cat*, syntactic labels like Det, phonological units like V...).
- Regular expressions (**regexes**) are used to specify **word patterns**.
- Regexes are built out of consist of **constants**, which denote sets of words, and **operator symbols**, which denote operations over these sets.
- Constants: $\emptyset$ (the empty regex), and $\{\epsilon\}$ (the empty-word regex) are regexes. For any w in Σ, $\{w\}$ is a regex.
- Operations: given R and S two regexes...
  - RS is the pairwise **concatenation** of R and S and is a regex.
  - $R|S$ is the **union** of R and S and is a regex.
  - R^* is the set containing ϵ and all possible concatenations of elements of R ("**Kleene star**") and is a regex.
- We'll use parentheses to signal the order of precedence between these operations.

A regex for intensified adjectival phrases

- Σ = { a, the, very, tall, girl, boy }.

A regex for intensified adjectival phrases

- Σ = { a, the, very, tall, girl, boy }.
- We want a regex to match the set of expressions {a boy, a girl, a tall boy, a tall girl, a very tall boy, a very tall girl, a very very tall boy, … the boy, the girl, …}.

A regex for intensified adjectival phrases

- Σ = { a, the, very, tall, girl, boy }.
- We want a regex to match the set of expressions {a boy, a girl, a tall boy, a tall girl, a very tall boy, a very tall girl, a very very tall boy, ... the boy, the girl, ...}.
- The phrase starts with *the* or with *a*.

A regex for intensified adjectival phrases

- Σ = { a, the, very, tall, girl, boy }.
- We want a regex to match the set of expressions {a boy, a girl, a tall boy, a tall girl, a very tall boy, a very tall girl, a very very tall boy, … the boy, the girl, …}.
- The phrase starts with *the* or with *a*. Achieved with the **union** operation (a|the).
- The phrase continues with an arbitrary number of *very* (including zero) followed by *tall*; or no adjective at all. The former option

A regex for intensified adjectival phrases

- Σ = { a, the, very, tall, girl, boy }.
- We want a regex to match the set of expressions {a boy, a girl, a tall boy, a tall girl, a very tall boy, a very tall girl, a very very tall boy, … the boy, the girl, …}.
- The phrase starts with *the* or with *a*. Achieved with the **union** operation (a|the).
- The phrase continues with an arbitrary number of *very* (including zero) followed by *tall*; or no adjective at all. The former option is modeled by **concatenating** the **Kleene star** of *very* (very*) with *tall* to form (very*)tall. The latter option

A regex for intensified adjectival phrases

- Σ = { a, the, very, tall, girl, boy }.
- We want a regex to match the set of expressions {a boy, a girl, a tall boy, a tall girl, a very tall boy, a very tall girl, a very very tall boy, ... the boy, the girl, ...}.
- The phrase starts with *the* or with *a*. Achieved with the **union** operation (a|the).
- The phrase continues with an arbitrary number of *very* (including zero) followed by *tall*; or no adjective at all. The former option is modeled by **concatenating** the **Kleene star** of *very* (very*) with *tall* to form (very*)tall. The latter option is simply ϵ (the empty word).

A regex for intensified adjectival phrases

- Σ = { a, the, very, tall, girl, boy }.
- We want a regex to match the set of expressions {a boy, a girl, a tall boy, a tall girl, a very tall boy, a very tall girl, a very very tall boy, ... the boy, the girl, ...}.
- The phrase starts with *the* or with *a*. Achieved with the **union** operation (a|the).
- The phrase continues with an arbitrary number of *very* (including zero) followed by *tall*; or no adjective at all. The former option is modeled by **concatenating** the **Kleene star** of *very* (very*) with *tall* to form (very*)tall. The latter option is simply ϵ (the empty word). We then union these two options ((very*)tall|ϵ).
- The phrase ends with *boy* or *girl*.

A regex for intensified adjectival phrases

- Σ = { a, the, very, tall, girl, boy }.
- We want a regex to match the set of expressions {a boy, a girl, a tall boy, a tall girl, a very tall boy, a very tall girl, a very very tall boy, ... the boy, the girl, ...}.
- The phrase starts with *the* or with *a*. Achieved with the **union** operation (a|the).
- The phrase continues with an arbitrary number of *very* (including zero) followed by *tall*; or no adjective at all. The former option is modeled by **concatenating** the **Kleene star** of *very* (very*) with *tall* to form (very*)tall. The latter option is simply ϵ (the empty word). We then union these two options ((very*)tall|ϵ).
- The phrase ends with *boy* or *girl*. Achieved again with the **union** operation (boy|girl).

A regex for intensified adjectival phrases

- Σ = { a, the, very, tall, girl, boy }.
- We want a regex to match the set of expressions {a boy, a girl, a tall boy, a tall girl, a very tall boy, a very tall girl, a very very tall boy, ... the boy, the girl, ...}.
- The phrase starts with *the* or with *a*. Achieved with the **union** operation (a|the).
- The phrase continues with an arbitrary number of *very* (including zero) followed by *tall*; or no adjective at all. The former option is modeled by **concatenating** the **Kleene star** of *very* (very*) with *tall* to form (very*)tall. The latter option is simply ϵ (the empty word). We then union these two options ((very*)tall|ϵ).
- The phrase ends with *boy* or *girl*. Achieved again with the **union** operation (boy|girl).
- **Concatenating** all these parts together form the desired pattern: (a|the)((very*)tall|ϵ)(boy|girl).
- What is the issue with the regex (a|the)(very*)(tall|ϵ)(boy|girl)?

Useful shorthands

- One can define useful **shorthands** from the three core operations (concatenation, union, Kleene star). If R and S are regexes:
  - R^+ is defined as RR^*: it is the set all possible concatenations of elements of R.
  - R^k with $k > 0$ a finite integer is defined as $\underbrace{RRR...R}_{k \text{ times}}$ is the set of all possible concatenations of elements of R involving exactly k elements.
  - $R^{\{k,l\}}$ with $l > k > 0$ finite integers is defined as $R^k|R^{k+1}|R^{k+2}|...|R^l$ is the set all possible concatenations of elements of R involving between k and l elements.
  - . (wildcard) stands for any element of the primitive alphabet Σ.
  - $R?$ is defined as $(R|\epsilon)$ and is the set of elements of R plus the empty word.

Useful shorthands

- One can define useful **shorthands** from the three core operations (concatenation, union, Kleene star). If R and S are regexes:
  - R^+ is defined as RR^*: it is the set all possible concatenations of elements of R.
  - R^k with $k > 0$ a finite integer is defined as $\underbrace{RRR...R}_{k \text{ times}}$ is the set of all possible concatenations of elements of R involving exactly k elements.
  - $R^{\{k,l\}}$ with $l > k > 0$ finite integers is defined as $R^k|R^{k+1}|R^{k+2}|...|R^l$ is the set all possible concatenations of elements of R involving between k and l elements.
  - . (wildcard) stands for any element of the primitive alphabet Σ.
  - $R?$ is defined as $(R|\epsilon)$ and is the set of elements of R plus the empty word.

Useful shorthands

- One can define useful **shorthands** from the three core operations (concatenation, union, Kleene star). If R and S are regexes:
  - R^+ is defined as RR^*: it is the set all possible concatenations of elements of R.
  - R^k with $k > 0$ a finite integer is defined as $\underbrace{RRR...R}_{k \text{ times}}$ is the set of all possible concatenations of elements of R involving exactly k elements.
  - $R^{\{k,l\}}$ with $l > k > 0$ finite integers is defined as $R^k|R^{k+1}|R^{k+2}|...|R^l$ is the set all possible concatenations of elements of R involving between k and l elements.
  - . (wildcard) stands for any element of the primitive alphabet Σ.
  - $R?$ is defined as $(R|\epsilon)$ and is the set of elements of R plus the empty word.

Useful shorthands

- One can define useful **shorthands** from the three core operations (concatenation, union, Kleene star). If R and S are regexes:
  - R^+ is defined as RR^*: it is the set all possible concatenations of elements of R.
  - R^k with $k > 0$ a finite integer is defined as $\underbrace{RRR...R}_{k \text{ times}}$ is the set of all possible concatenations of elements of R involving exactly k elements.
  - $\mathrm{R}^{\{k,l\}}$ with $l > k > 0$ finite integers is defined as $R^k|R^{k+1}|R^{k+2}|...|R^l$ is the set all possible concatenations of elements of R involving between k and l elements.
  - . (wildcard) stands for any element of the primitive alphabet Σ.
  - R? is defined as $(R|\epsilon)$ and is the set of elements of R plus the empty word.

Useful shorthands

- One can define useful **shorthands** from the three core operations (concatenation, union, Kleene star). If R and S are regexes:
  - R^+ is defined as RR^*: it is the set all possible concatenations of elements of R.
  - R^k with $k > 0$ a finite integer is defined as $\underbrace{RRR...R}_{k \text{ times}}$ is the set of all possible concatenations of elements of R involving exactly k elements.
  - $\mathrm{R}^{\{k,l\}}$ with $l > k > 0$ finite integers is defined as $R^k|R^{k+1}|R^{k+2}|...|R^l$ is the set all possible concatenations of elements of R involving between k and l elements.
  - . (wildcard) stands for any element of the primitive alphabet Σ.
  - R? is defined as $(R|\epsilon)$ and is the set of elements of R plus the empty word.

Useful shorthands

- One can define useful **shorthands** from the three core operations (concatenation, union, Kleene star). If R and S are regexes:
  - R^+ is defined as RR^*: it is the set all possible concatenations of elements of R.
  - R^k with $k > 0$ a finite integer is defined as $\underbrace{RRR...R}_{k \text{ times}}$ is the set of all possible concatenations of elements of R involving exactly k elements.
  - $\mathrm{R}^{\{k,l\}}$ with $l > k > 0$ finite integers is defined as $R^k|R^{k+1}|R^{k+2}|...|R^l$ is the set all possible concatenations of elements of R involving between k and l elements.
  - . (wildcard) stands for any element of the primitive alphabet Σ.
  - R? is defined as $(R|\epsilon)$ and is the set of elements of R plus the empty word.

Practice: right-embedded complementation

- Suppose $\Sigma = \{\text{NP, V, VP, that}\}$.

Practice: right-embedded complementation

- Suppose $\Sigma = \{\text{NP, V, VP, that}\}$.
- We want to model sentences of the form *Jo believes that Al thinks … that Lu imagines that Ed is happy*, where *Jo*, *Al*, *Lu*, …, *Ed* are NPs, *believes*, *thinks*, …, *imagines* are V, and *is happy* is a VP.
- What's the pattern of *Ed is happy*?

Practice: right-embedded complementation

- Suppose $\Sigma = \{\text{NP, V, VP, that}\}$.
- We want to model sentences of the form *Jo believes that Al thinks … that Lu imagines that Ed is happy*, where *Jo*, *Al*, *Lu*, …, *Ed* are NPs, *believes*, *thinks*, …, *imagines* are V, and *is happy* is a VP.
- What's the pattern of *Ed is happy*? NP VP.
- Of *Lu imagines that Ed is happy*?

Practice: right-embedded complementation

- Suppose $\Sigma = \{\text{NP, V, VP, that}\}$.
- We want to model sentences of the form *Jo believes that Al thinks … that Lu imagines that Ed is happy*, where *Jo*, *Al*, *Lu*, …, *Ed* are NPs, *believes*, *thinks*, …, *imagines* are V, and *is happy* is a VP.
- What's the pattern of *Ed is happy*? NP VP.
- Of *Lu imagines that Ed is happy*? NP V that NP VP.
- Of *Al thinks that Lu imagines that Ed is happy*?

Practice: right-embedded complementation

- Suppose $\Sigma = \{\text{NP, V, VP, that}\}$.
- We want to model sentences of the form *Jo believes that Al thinks … that Lu imagines that Ed is happy*, where *Jo*, *Al*, *Lu*, …, *Ed* are NPs, *believes*, *thinks*, …, *imagines* are V, and *is happy* is a VP.
- What's the pattern of *Ed is happy*? NP VP.
- Of *Lu imagines that Ed is happy*? NP V that NP VP.
- Of *Al thinks that Lu imagines that Ed is happy*? NP V that NP V that NP VP.
- Of *Jo believes that Al thinks that Lu imagines that Ed is happy*?

Practice: right-embedded complementation

- Suppose $\Sigma = \{\text{NP, V, VP, that}\}$.
- We want to model sentences of the form *Jo believes that Al thinks … that Lu imagines that Ed is happy*, where *Jo*, *Al*, *Lu*, …, *Ed* are NPs, *believes*, *thinks*, …, *imagines* are V, and *is happy* is a VP.
- What's the pattern of *Ed is happy*? NP VP.
- Of *Lu imagines that Ed is happy*? NP V that NP VP.
- Of *Al thinks that Lu imagines that Ed is happy*? NP V that NP V that NP VP.
- Of *Jo believes that Al thinks that Lu imagines that Ed is happy*? NP V that NP V that NP V that NP VP.
- The general pattern is: NP V that 0 or more times, followed by NP VP.

Practice: right-embedded complementation

- Suppose $\Sigma = \{\text{NP, V, VP, that}\}$.
- We want to model sentences of the form *Jo believes that Al thinks … that Lu imagines that Ed is happy*, where *Jo*, *Al*, *Lu*, …, *Ed* are NPs, *believes*, *thinks*, …, *imagines* are V, and *is happy* is a VP.
- What's the pattern of *Ed is happy*? NP VP.
- Of *Lu imagines that Ed is happy*? NP V that NP VP.
- Of *Al thinks that Lu imagines that Ed is happy*? NP V that NP V that NP VP.
- Of *Jo believes that Al thinks that Lu imagines that Ed is happy*? NP V that NP V that NP V that NP VP.
- The general pattern is: NP V that 0 or more times, followed by NP VP.
- Regex:

Practice: right-embedded complementation

- Suppose $\Sigma = \{\text{NP, V, VP, that}\}$.
- We want to model sentences of the form *Jo believes that Al thinks … that Lu imagines that Ed is happy*, where *Jo*, *Al*, *Lu*, …, *Ed* are NPs, *believes*, *thinks*, …, *imagines* are V, and *is happy* is a VP.
- What's the pattern of *Ed is happy*? NP VP.
- Of *Lu imagines that Ed is happy*? NP V that NP VP.
- Of *Al thinks that Lu imagines that Ed is happy*? NP V that NP V that NP VP.
- Of *Jo believes that Al thinks that Lu imagines that Ed is happy*? NP V that NP V that NP V that NP VP.
- The general pattern is: NP V that 0 or more times, followed by NP VP.
- Regex: (NP V that)*(NP VP)

Applications

- Mining a dataset for specific verbs and all their **possible inflections** (including noisy/erroneous ones!), for instance CHILDES when studying the acquisition of verbal predicates.
- Mining a naturalistic corpus for a wide range of **contracted/colloquial forms**; for instance when studying language change or sociolingusitic variation.
- Looking for specific **coocurrences separated by some arbitrary material**, e.g. perception verbs followed at some point by gerunds (*I see the girl with an umbrella dancing*); adjectival predicates followed by infinitives...
- Elaborate **PoS-tagging/parsing** based on morphological cues (e.g. suffixes *-ation*/*-ational*/*-ationalization*...).
- Automatic and conditional "**find-and-replace**" based on a pattern rather than one single string. See the Unix regex-based commands `grep` (find) and `sed` (find-and-replace).

Applications

- Mining a dataset for specific verbs and all their **possible inflections** (including noisy/erroneous ones!), for instance CHILDES when studying the acquisition of verbal predicates.
- Mining a naturalistic corpus for a wide range of **contracted/colloquial forms**; for instance when studying language change or sociolingusitic variation.
- Looking for specific **coocurrences separated by some arbitrary material**, e.g. perception verbs followed at some point by gerunds (*I see the girl with an umbrella dancing*); adjectival predicates followed by infinitives...
- Elaborate **PoS-tagging/parsing** based on morphological cues (e.g. suffixes *-ation*/*-ational*/*-ationalization*...).
- Automatic and conditional "**find-and-replace**" based on a pattern rather than one single string. See the Unix regex-based commands `grep` (find) and `sed` (find-and-replace).

Applications

- Mining a dataset for specific verbs and all their **possible inflections** (including noisy/erroneous ones!), for instance CHILDES when studying the acquisition of verbal predicates.
- Mining a naturalistic corpus for a wide range of **contracted/colloquial forms**; for instance when studying language change or sociolingusitic variation.
- Looking for specific **coocurrences separated by some arbitrary material**, e.g. perception verbs followed at some point by gerunds (*I see the girl with an umbrella dancing*); adjectival predicates followed by infinitives...
- Elaborate **PoS-tagging/parsing** based on morphological cues (e.g. suffixes *-ation*/*-ational*/*-ationalization*...).
- Automatic and conditional "**find-and-replace**" based on a pattern rather than one single string. See the Unix regex-based commands `grep` (find) and `sed` (find-and-replace).

Applications

- Mining a dataset for specific verbs and all their **possible inflections** (including noisy/erroneous ones!), for instance CHILDES when studying the acquisition of verbal predicates.
- Mining a naturalistic corpus for a wide range of **contracted/colloquial forms**; for instance when studying language change or sociolingusitic variation.
- Looking for specific **coocurrences separated by some arbitrary material**, e.g. perception verbs followed at some point by gerunds (*I see the girl with an umbrella dancing*); adjectival predicates followed by infinitives...
- Elaborate **PoS-tagging/parsing** based on morphological cues (e.g. suffixes *-ation*/*-ational*/*-ationalization*...).
- Automatic and conditional "**find-and-replace**" based on a pattern rather than one single string. See the Unix regex-based commands `grep` (find) and `sed` (find-and-replace).

Applications

- Mining a dataset for specific verbs and all their **possible inflections** (including noisy/erroneous ones!), for instance CHILDES when studying the acquisition of verbal predicates.
- Mining a naturalistic corpus for a wide range of **contracted/colloquial forms**; for instance when studying language change or sociolingusitic variation.
- Looking for specific **coocurrences separated by some arbitrary material**, e.g. perception verbs followed at some point by gerunds (*I see the girl with an umbrella dancing*); adjectival predicates followed by infinitives...
- Elaborate **PoS-tagging/parsing** based on morphological cues (e.g. suffixes *-ation*/*-ational*/*-ationalization*...).
- Automatic and conditional "**find-and-replace**" based on a pattern rather than one single string. See the Unix regex-based commands `grep` (find) and `sed` (find-and-replace).

Graphical representation: automata

- Regexes can be represented using diagrams called **automata**.
- Automata have **states**, represented with **circles**. There's one **initial** state (where you start), and at least one **accepting** state (where you want to end, signaled with a double circle).
- States are connected with arrows labeled with words of the alphabet Σ (including the empty word ϵ).
- An automata **matches a sentence** if there is a path from the initial state to an accepting state whose successive arrows match the words of the sentence.

Graphical representation: automata

- Regexes can be represented using diagrams called **automata**.
- Automata have **states**, represented with **circles**. There's one **initial** state (where you start), and at least one **accepting** state (where you want to end, signaled with a double circle).
- States are connected with arrows labeled with words of the alphabet Σ (including the empty word ϵ).
- An automata **matches a sentence** if there is a path from the initial state to an accepting state whose successive arrows match the words of the sentence.

Graphical representation: automata

- Regexes can be represented using diagrams called **automata**.
- Automata have **states**, represented with **circles**. There's one **initial** state (where you start), and at least one **accepting** state (where you want to end, signaled with a double circle).
- States are connected with arrows labeled with words of the alphabet Σ (including the empty word ϵ).
- An automata **matches a sentence** if there is a path from the initial state to an accepting state whose successive arrows match the words of the sentence.

Graphical representation: automata

- Regexes can be represented using diagrams called **automata**.
- Automata have **states**, represented with **circles**. There's one **initial** state (where you start), and at least one **accepting** state (where you want to end, signaled with a double circle).
- States are connected with arrows labeled with words of the alphabet Σ (including the empty word ϵ).
- An automata **matches a sentence** if there is a path from the initial state to an accepting state whose successive arrows match the words of the sentence.

Concatenation automaton

- The following automaton accepts only one sentence, *the boy*. It corresponds to the concatenated regex the boy.



- More generally, regex **concatenation** is represented by **arrows between states**.

Concatenation automaton

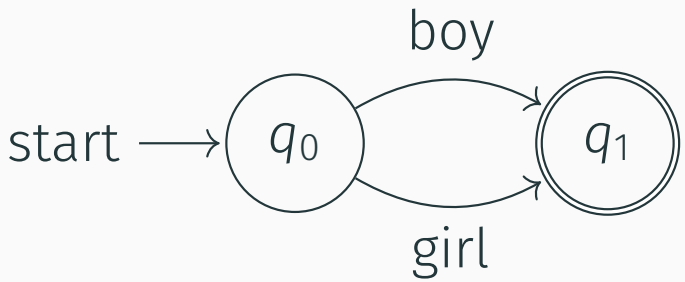
- The following automaton accepts only one sentence, *the boy*. It corresponds to the concatenated regex the boy.



- More generally, regex **concatenation** is represented by **arrows between states**.

Union automaton

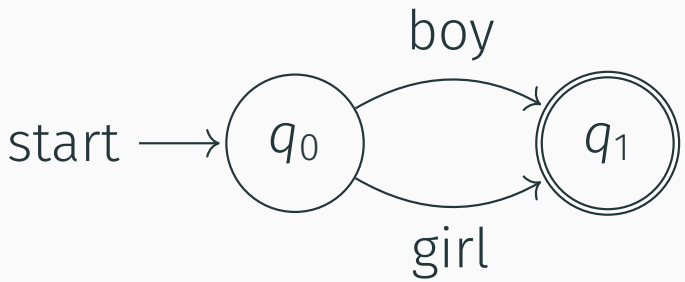
- The following automaton accepts *boy* or *girl*, i.e. it corresponds to the regex union operation (boy|girl).



- More generally, regex **union** is represented by **multiple arrows** originating from the same state.

Union automaton

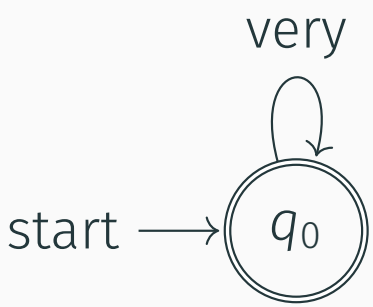
- The following automaton accepts *boy* or *girl*, i.e. it corresponds to the regex union operation (boy|girl).



- More generally, regex **union** is represented by **multiple arrows** originating from the same state.

Kleene star automaton

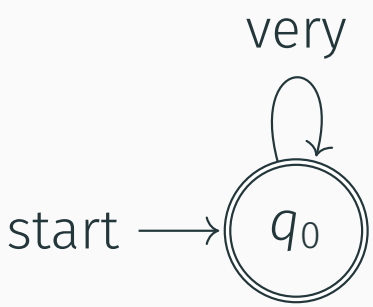
- The following automaton accepts ϵ, *very*, *very very*, *very very very*...i.e. it corresponds to (*very**).



- More generally, **Kleene star** operations are **feedback loops** to "previous" states.

Kleene star automaton

- The following automaton accepts ϵ, *very*, *very very*, *very very very*...i.e. it corresponds to (*very**).



- More generally, **Kleene star** operations are **feedback loops** to "previous" states.

Practice: automaton for intensified adjectival phrases

- Recall that our pattern for *the/a boy/girl*, *the/a tall boy/girl*, *the/a very tall boy/girl*... was: (a|the)((very*)tall|ϵ)(boy|girl).

Practice: automaton for intensified adjectival phrases

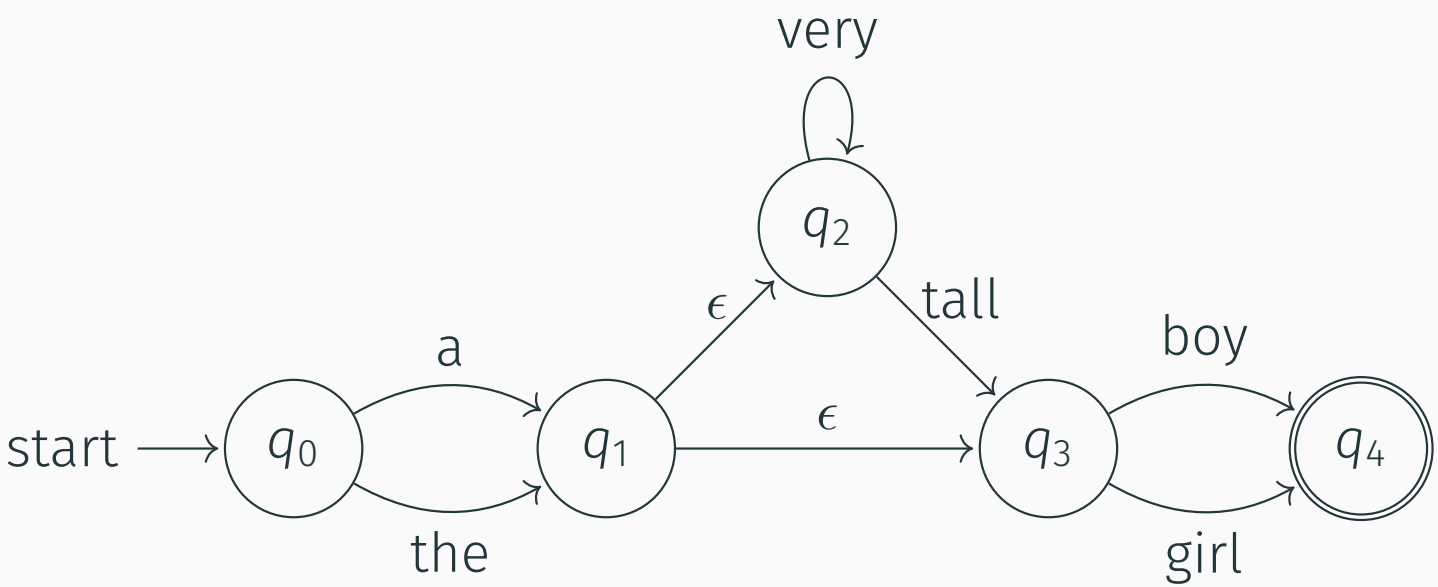
- Recall that our pattern for *the/a boy/girl*, *the/a tall boy/girl*, *the/a very tall boy/girl*... was: (a|the)((very*)tall|ϵ)(boy|girl).
- It meant that first we want a determiner (*a* or *the*), then we want either an optionally intensified adjective (*very very ... very tall*) or nothing, and lastly, we want a noun (*boy* or *girl*).

Practice: automaton for intensified adjectival phrases

- Recall that our pattern for *the/a boy/girl*, *the/a tall boy/girl*, *the/a very tall boy/girl*... was: (a|the)((very*)tall|ϵ)(boy|girl).
- It meant that first we want a determiner (*a* or *the*), then we want either an optionally intensified adjective (*very very ... very tall*) or nothing, and lastly, we want a noun (*boy* or *girl*).
- What's the automaton for this?

Practice: automaton for intensified adjectival phrases

- Recall that our pattern for *the/a boy/girl*, *the/a tall boy/girl*, *the/a very tall boy/girl*... was: (a|the)((very*)tall|ϵ)(boy|girl).
- It meant that first we want a determiner (*a* or *the*), then we want either an optionally intensified adjective (*very very ... very tall*) or nothing, and lastly, we want a noun (*boy* or *girl*).
- What's the automaton for this?



Practice: automaton for right-embedded complementation

- Recall that our pattern for *Jo believes that Al thinks… that Ed is happy…* was: (NP V that)*(NP VP).

Practice: automaton for right-embedded complementation

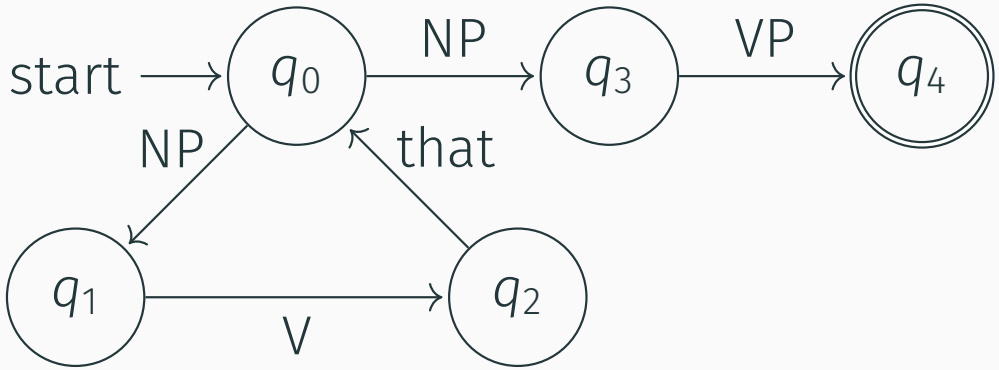
- Recall that our pattern for *Jo believes that Al thinks… that Ed is happy…* was: (NP V that)*(NP VP).
- It meant that first we want NP V that (an embedding sentence) zero or more times, and then we finish by NP VP (the last embedded sentence).

Practice: automaton for right-embedded complementation

- Recall that our pattern for *Jo believes that Al thinks… that Ed is happy…* was: (NP V that)*(NP VP).
- It meant that first we want NP V that (an embedding sentence) zero or more times, and then we finish by NP VP (the last embedded sentence).
- What's the automaton for this?

Practice: automaton for right-embedded complementation

- Recall that our pattern for *Jo believes that Al thinks… that Ed is happy…* was: (NP V that)*(NP VP).
- It meant that first we want NP V that (an embedding sentence) zero or more times, and then we finish by NP VP (the last embedded sentence).
- What's the automaton for this?



Linguistic challenges for regular expressions and automata

Structural ambiguities

- Compare:

(8) a. Tall boys and girls.
b. Tall boys and tall girls.
c. Tall boys and short girls.
d. Boys and tall girls.

- When an adjective precedes conjoined NPs, as in (8a), **it's ambiguous whether the adjective applies to both NPs** (as in (8b)), **or to the first NP only** (with a meaning compatible with (8c)).
- This ambiguity disappears if the adjective follows the first and precedes the second NP, as in (8d).
- Similar-ish ambiguities arise in sentences like (9) and (10).

(9) The photographer saw the bird on the hill.

(10) They are building blocks.

Structural ambiguities

- Compare:

(8) a. Tall boys and girls.
b. Tall boys and tall girls.
c. Tall boys and short girls.
d. Boys and tall girls.

- When an adjective precedes conjoined NPs, as in (8a), **it's ambiguous whether the adjective applies to both NPs** (as in (8b)), **or to the first NP only** (with a meaning compatible with (8c)).
- This ambiguity disappears if the adjective follows the first and precedes the second NP, as in (8d).
- Similar-ish ambiguities arise in sentences like (9) and (10).

(9) The photographer saw the bird on the hill.

(10) They are building blocks.

Structural ambiguities

- Compare:

(8) a. Tall boys and girls.
b. Tall boys and tall girls.
c. Tall boys and short girls.
d. Boys and tall girls.

- When an adjective precedes conjoined NPs, as in (8a), **it's ambiguous whether the adjective applies to both NPs** (as in (8b)), **or to the first NP only** (with a meaning compatible with (8c)).
- This ambiguity disappears if the adjective follows the first and precedes the second NP, as in (8d).
- Similar-ish ambiguities arise in sentences like (9) and (10).

(9) The photographer saw the bird on the hill.

(10) They are building blocks.

Structural ambiguities

- Compare:

(8) a. Tall boys and girls.
b. Tall boys and tall girls.
c. Tall boys and short girls.
d. Boys and tall girls.

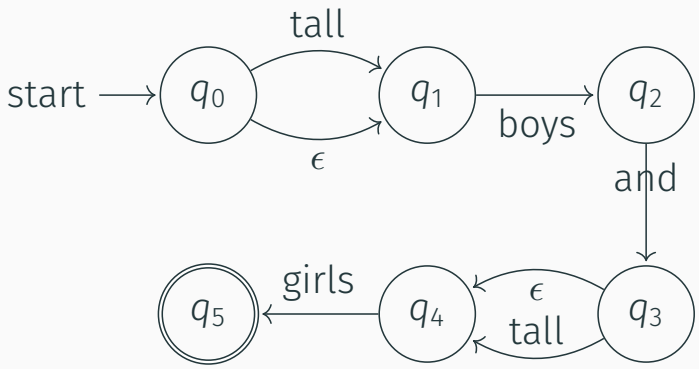
- When an adjective precedes conjoined NPs, as in (8a), **it's ambiguous whether the adjective applies to both NPs** (as in (8b)), **or to the first NP only** (with a meaning compatible with (8c)).
- This ambiguity disappears if the adjective follows the first and precedes the second NP, as in (8d).
- Similar-ish ambiguities arise in sentences like (9) and (10).

(9) The photographer saw the bird on the hill.

(10) They are building blocks.

Regexes/automata have a “weak” generative capacity

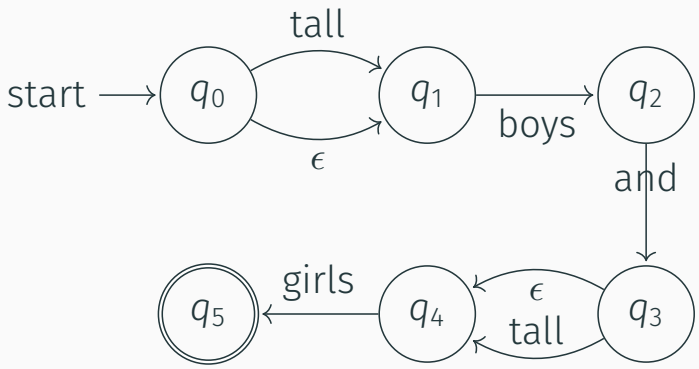
- Regexes/automata *can* match strings like *tall boys and girls*, *tall boys and tall girls* etc: (tall|ϵ) boys and (tall|ϵ) girls.



- But nothing in the traversal of the automata, or in the denotation of the regex (a sets of strings!) tells us that *tall boys and girls* can be understood either as *[tall [boys]] and [girls]* or as *[tall [boys and girls]]*!
- These devices capture the well-formedness of the string, but **nothing about its internal structure** (which influences meaning).
- In that sense, regexes and automata have a **weak generative capacity** w.r.t. structural ambiguities.

Regexes/automata have a “weak” generative capacity

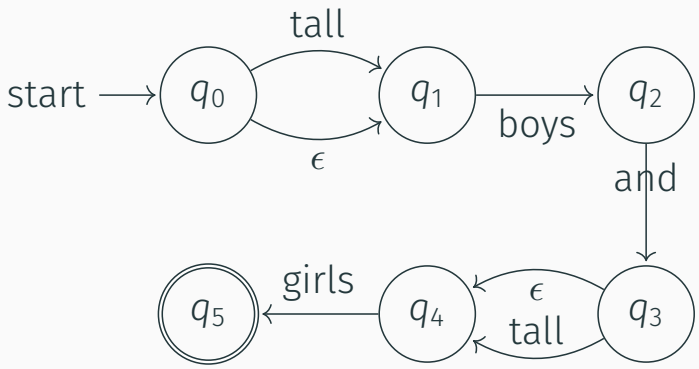
- Regexes/automata *can* match strings like *tall boys and girls*, *tall boys and tall girls* etc: (tall|ϵ) boys and (tall|ϵ) girls.



- But nothing in the traversal of the automata, or in the denotation of the regex (a sets of strings!) tells us that *tall boys and girls* can be understood either as *[tall [boys]] and [girls]* or as *[tall [boys and girls]]*!
- These devices capture the well-formedness of the string, but **nothing about its internal structure** (which influences meaning).
- In that sense, regexes and automata have a **weak generative capacity** w.r.t. structural ambiguities.

Regexes/automata have a “weak” generative capacity

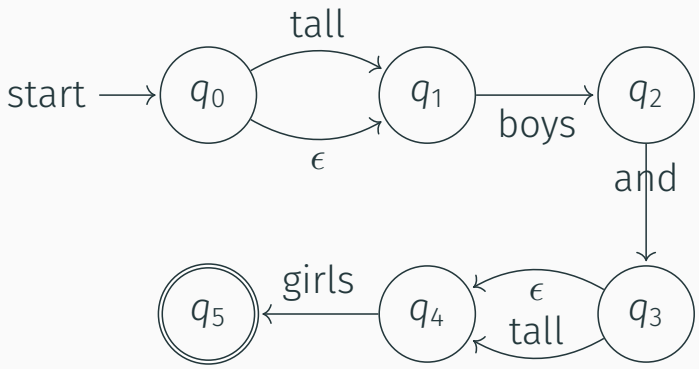
- Regexes/automata *can* match strings like *tall boys and girls*, *tall boys and tall girls* etc: (tall|ϵ) boys and (tall|ϵ) girls.



- But nothing in the traversal of the automata, or in the denotation of the regex (a sets of strings!) tells us that *tall boys and girls* can be understood either as *[tall [boys]] and [girls]* or as *[tall [boys and girls]]*!
- These devices capture the well-formedness of the string, but **nothing about its internal structure** (which influences meaning).
- In that sense, regexes and automata have a **weak generative capacity** w.r.t. structural ambiguities.

Regexes/automata have a “weak” generative capacity

- Regexes/automata *can* match strings like *tall boys and girls*, *tall boys and tall girls* etc: (tall|ϵ) boys and (tall|ϵ) girls.



- But nothing in the traversal of the automata, or in the denotation of the regex (a sets of strings!) tells us that *tall boys and girls* can be understood either as *[tall [boys]] and [girls]* or as *[tall [boys and girls]]*!
- These devices capture the well-formedness of the string, but **nothing about its internal structure** (which influences meaning).
- In that sense, regexes and automata have a **weak generative capacity** w.r.t. structural ambiguities.

Deeper challenge: center-embedding

- Let's consider nested subject-modifying, object-gap relative clauses:

(11) a. The mouse is scared.

b. The mouse [that the cat chases] is scared.

c. The mouse [that the cat [that the dog barks at] chases] is scared.

d. The mouse [that the cat [that the dog [that the postman avoids] barks at] chases] is scared.

- The corresponding structures are: NP VP; NP that NP V VP; NP that NP that NP V V VP; NP that NP that NP that NP V V V VP... etc.
- Can we synthesize this using one single regex?
- The pattern appears to be: NP (that NP)^k (V)^k VP, **for any** $k \geq 0$. But there are an infinity of such k, so our "regex" would need to be infinitely long... Not a regex!

Deeper challenge: center-embedding

- Let's consider nested subject-modifying, object-gap relative clauses:

(11) a. The mouse is scared.

b. The mouse [that the cat chases] is scared.

c. The mouse [that the cat [that the dog barks at] chases] is scared.

d. The mouse [that the cat [that the dog [that the postman avoids] barks at] chases] is scared.

- The corresponding structures are: NP VP; NP that NP V VP; NP that NP that NP V V VP; NP that NP that NP that NP V V V VP... etc.
- Can we synthesize this using one single regex?
- The pattern appears to be: NP (that NP)^k (V)^k VP, **for any** $k \geq 0$. But there are an infinity of such k, so our "regex" would need to be infinitely long... Not a regex!

Deeper challenge: center-embedding

- Let's consider nested subject-modifying, object-gap relative clauses:

(11) a. The mouse is scared.

b. The mouse [that the cat chases] is scared.

c. The mouse [that the cat [that the dog barks at] chases] is scared.

d. The mouse [that the cat [that the dog [that the postman avoids] barks at] chases] is scared.

- The corresponding structures are: NP VP; NP that NP V VP; NP that NP that NP V V VP; NP that NP that NP that NP V V V VP... etc.
- Can we synthesize this using one single regex?
- The pattern appears to be: NP (that NP)^k (V)^k VP, **for any** $k \geq 0$. But there are an infinity of such k, so our "regex" would need to be infinitely long... Not a regex!

Deeper challenge: center-embedding

- Let's consider nested subject-modifying, object-gap relative clauses:

(11) a. The mouse is scared.

b. The mouse [that the cat chases] is scared.

c. The mouse [that the cat [that the dog barks at] chases] is scared.

d. The mouse [that the cat [that the dog [that the postman avoids] barks at] chases] is scared.

- The corresponding structures are: NP VP; NP that NP V VP; NP that NP that NP V V VP; NP that NP that NP that NP V V V VP... etc.
- Can we synthesize this using one single regex?
- The pattern appears to be: NP (that NP)^k (V)^k VP, **for any** $k \geq 0$. But there are an infinity of such k, so our "regex" would need to be infinitely long... Not a regex!

Deeper challenge: center-embedding

- Let's consider nested subject-modifying, object-gap relative clauses:

(11) a. The mouse is scared.

b. The mouse [that the cat chases] is scared.

c. The mouse [that the cat [that the dog barks at] chases] is scared.

d. The mouse [that the cat [that the dog [that the postman avoids] barks at] chases] is scared.

- The corresponding structures are: NP VP; NP that NP V VP; NP that NP that NP V V VP; NP that NP that NP that NP V V V VP... etc.
- Can we synthesize this using one single regex?
- The pattern appears to be: NP (that NP)^k (V)^k VP, **for any** $k \geq 0$. But there are an infinity of such k, so our "regex" would need to be infinitely long... Not a regex!

Deeper challenge: center-embedding

- Let's consider nested subject-modifying, object-gap relative clauses:

(11) a. The mouse is scared.
b. The mouse [that the cat chases] is scared.
c. The mouse [that the cat [that the dog barks at] chases] is scared.
d. The mouse [that the cat [that the dog [that the postman avoids] barks at] chases] is scared.

- The corresponding structures are: NP VP; NP that NP V VP; NP that NP that NP V V VP; NP that NP that NP that NP V V V VP... etc.
- Can we synthesize this using one single regex?
- The pattern appears to be: NP (that NP)^k (V)^k VP, **for any** $k \geq 0$. But there are an infinity of such k, so our "regex" would need to be infinitely long... Not a regex!

Deeper challenge: center-embedding

- Let's consider nested subject-modifying, object-gap relative clauses:

(11) a. The mouse is scared.
b. The mouse [that the cat chases] is scared.
c. The mouse [that the cat [that the dog barks at] chases] is scared.
d. The mouse [that the cat [that the dog [that the postman avoids] barks at] chases] is scared.

- The corresponding structures are: NP VP; NP that NP V VP; NP that NP that NP V V VP; NP that NP that NP that NP V V V VP... etc.
- Can we synthesize this using one single regex?
- The pattern appears to be: NP (that NP)^k (V)^k VP, **for any** $k \geq 0$. But there are an infinity of such k, so our "regex" would need to be infinitely long... Not a regex!

Deeper challenge: center-embedding

- Let's consider nested subject-modifying, object-gap relative clauses:

(11) a. The mouse is scared.
b. The mouse [that the cat chases] is scared.
c. The mouse [that the cat [that the dog barks at] chases] is scared.
d. The mouse [that the cat [that the dog [that the postman avoids] barks at] chases] is scared.

- The corresponding structures are: NP VP; NP that NP V VP; NP that NP that NP V V VP; NP that NP that NP that NP V V V VP... etc.
- Can we synthesize this using one single regex?
- The pattern appears to be: NP (that NP)^k (V)^k VP, **for any** $k \geq 0$. But there are an infinity of such k, so our "regex" would need to be infinitely long... Not a regex!

Challenges summary

- The fundamental limitation of regexes/automata is that they **cannot keep track of arbitrary occurrence counts**. Anything that looks like a sequence of k identical patterns, followed by a sequence of k (or $k-1$, or $k+6$...) other patterns, cannot be matched by regexes/automata. They are **not expressive enough**.
- This sheds light on a **fundamental complexity difference between right-embedding** (*Jo thinks that Ed believes that...*), which *can* be captured by regexes, **and center-embedding** (*The mouse that the cat that... chases is scared*), which cannot.
- We have also seen that regexes and automata **cannot account for the internal structure of strings**, and the meaning differences that may arise from structural ambiguities.
- We'll introduce context-free grammars next week to encode more structure and recursivity.

Challenges summary

- The fundamental limitation of regexes/automata is that they **cannot keep track of arbitrary occurrence counts**. Anything that looks like a sequence of k identical patterns, followed by a sequence of k (or $k-1$, or $k+6$...) other patterns, cannot be matched by regexes/automata. They are **not expressive enough**.
- This sheds light on a **fundamental complexity difference between right-embedding** (*Jo thinks that Ed believes that...*), which *can* be captured by regexes, **and center-embedding** (*The mouse that the cat that... chases is scared*), which cannot.
- We have also seen that regexes and automata **cannot account for the internal structure of strings**, and the meaning differences that may arise from structural ambiguities.
- We'll introduce context-free grammars next week to encode more structure and recursivity.

Challenges summary

- The fundamental limitation of regexes/automata is that they **cannot keep track of arbitrary occurrence counts**. Anything that looks like a sequence of k identical patterns, followed by a sequence of k (or $k-1$, or $k+6$...) other patterns, cannot be matched by regexes/automata. They are **not expressive enough**.
- This sheds light on a **fundamental complexity difference between right-embedding** (*Jo thinks that Ed believes that...*), which *can* be captured by regexes, **and center-embedding** (*The mouse that the cat that... chases is scared*), which cannot.
- We have also seen that regexes and automata **cannot account for the internal structure of strings**, and the meaning differences that may arise from structural ambiguities.
- We'll introduce context-free grammars next week to encode more structure and recursivity.

Challenges summary

- The fundamental limitation of regexes/automata is that they **cannot keep track of arbitrary occurrence counts**. Anything that looks like a sequence of k identical patterns, followed by a sequence of k (or $k-1$, or $k+6$...) other patterns, cannot be matched by regexes/automata. They are **not expressive enough**.
- This sheds light on a **fundamental complexity difference between right-embedding** (*Jo thinks that Ed believes that...*), which *can* be captured by regexes, **and center-embedding** (*The mouse that the cat that... chases is scared*), which cannot.
- We have also seen that regexes and automata **cannot account for the internal structure of strings**, and the meaning differences that may arise from structural ambiguities.
- We'll introduce context-free grammars next week to encode more structure and recursivity.

Lab

https://colab.research.google.com/drive/1khF9SnSJbzUVMIb5Ws-CoOZ6SFH6u2cGv?usp=sharing